



**Team
Rutgers**



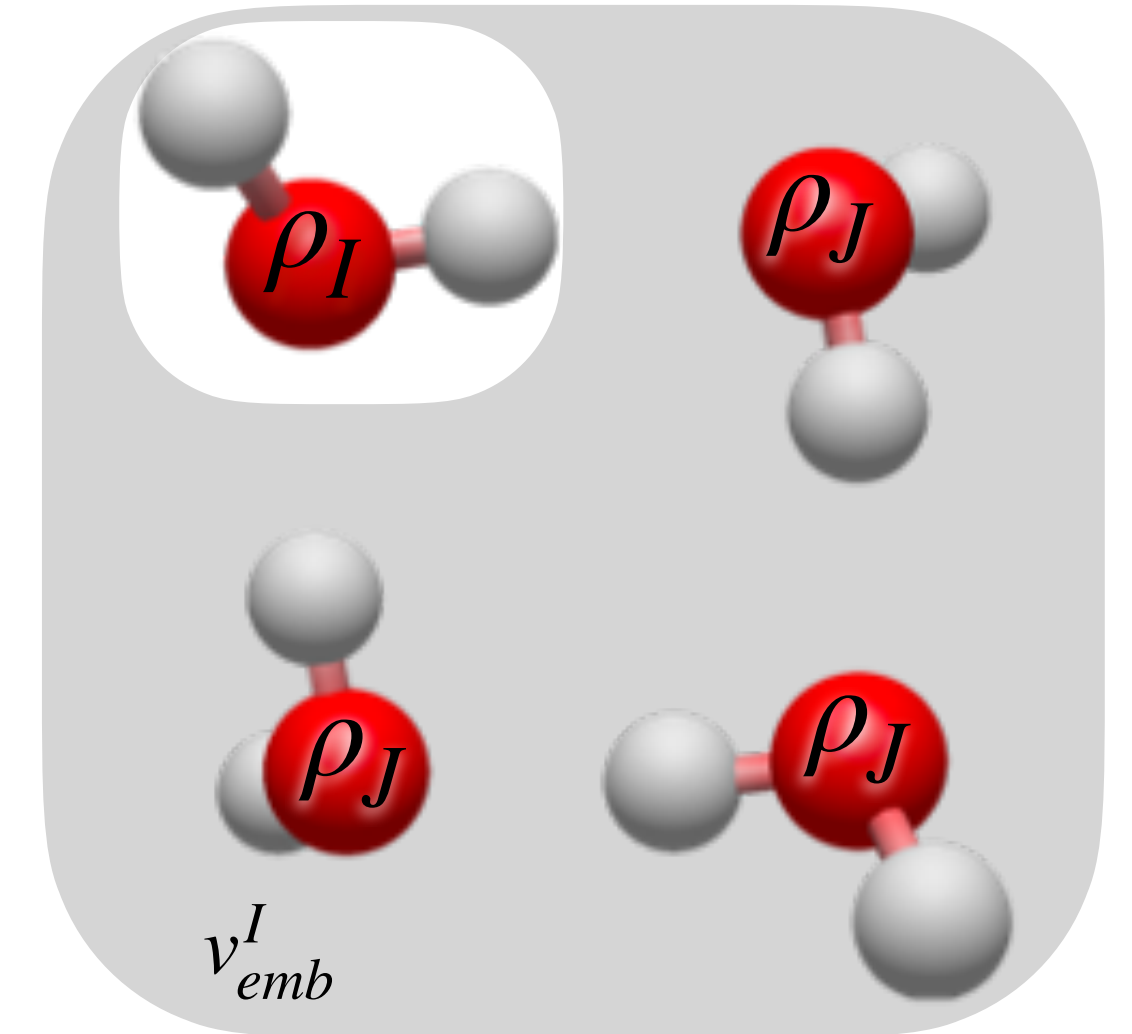
Embedding Electronic Structure Methods

sDFT: Divide and Conquer!

- Split system into (smaller) subsystems: $\rho(\mathbf{r}) = \sum_{I=1}^{N_S} \rho_I(\mathbf{r})$

- Total Energy: $E[\{\rho_I\}] = \sum_I E_I[\rho_I] + E^{nad}[\{\rho_I\}]$

- Set of KS equations: $\left[\frac{-\nabla_i^2}{2} + v_s^I[\rho_I](\mathbf{r}) + v_{emb}^I[\rho_I, \rho](\mathbf{r}) \right] \psi_i^I(\mathbf{r}) = \epsilon_i^I(\mathbf{r}) \psi_i^I(\mathbf{r})$



$$v_s^I[\rho_I](\mathbf{r}) = v_{ext}^I(\mathbf{r}) + v_H[\rho_I](\mathbf{r}) + v_{XC}[\rho_I](\mathbf{r})$$

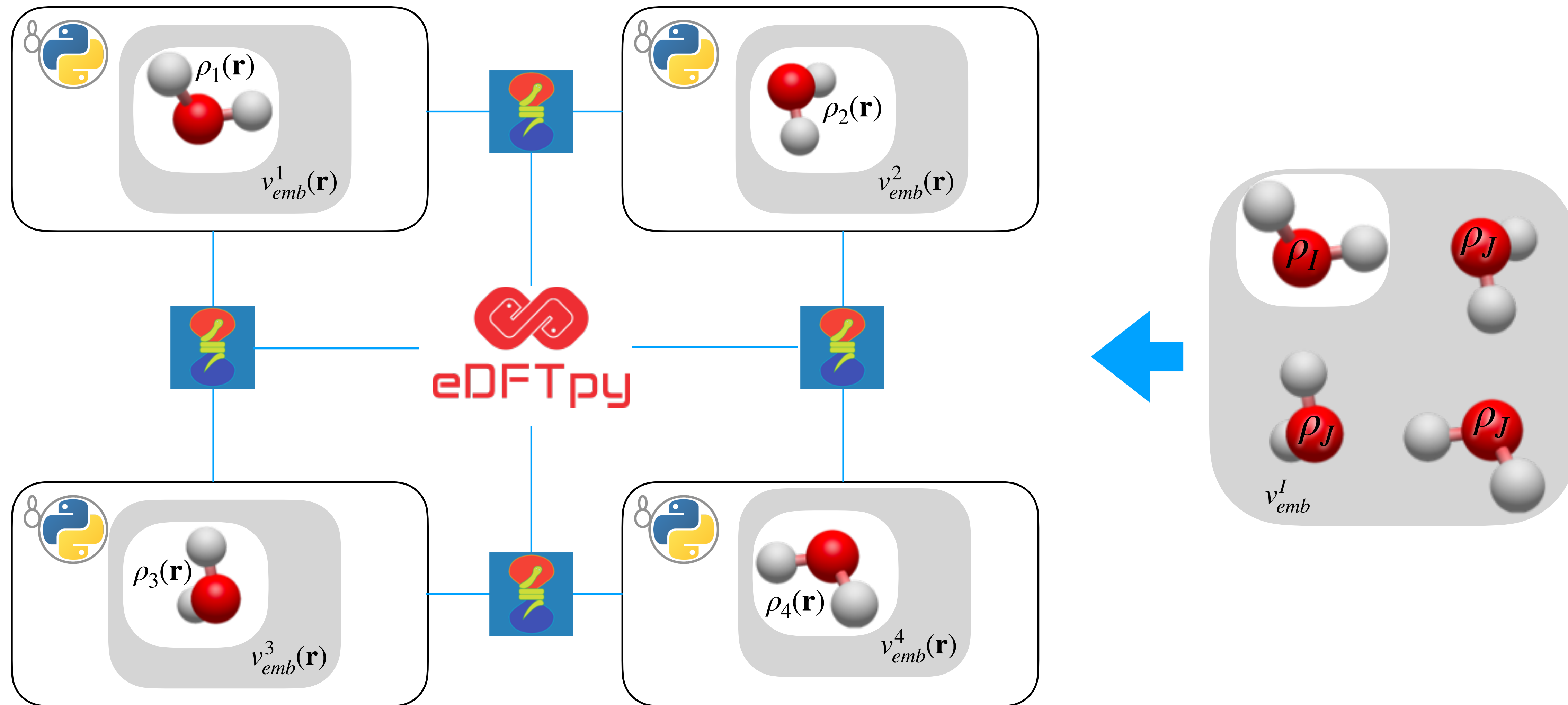
$$v_{emb}^I[\rho_I, \rho](\mathbf{r}) = \left(\sum_{J, J \neq I}^{N_S} v_{ext}^J(\mathbf{r}) + v_H[\rho_J](\mathbf{r}) \right) + \frac{\delta T_s[\rho]}{\delta \rho(\mathbf{r})} + \frac{\delta T_s[\rho_I]}{\delta \rho_I(\mathbf{r})} + \frac{\delta E_{xc}[\rho]}{\delta \rho(\mathbf{r})} + \frac{\delta E_{xc}[\rho_I]}{\delta \rho_I(\mathbf{r})}$$

$$v_{emb, I}^H[\{\rho_I\}](\mathbf{r})$$

$$v_{Ts}^{nad}[\{\rho_I\}](\mathbf{r})$$

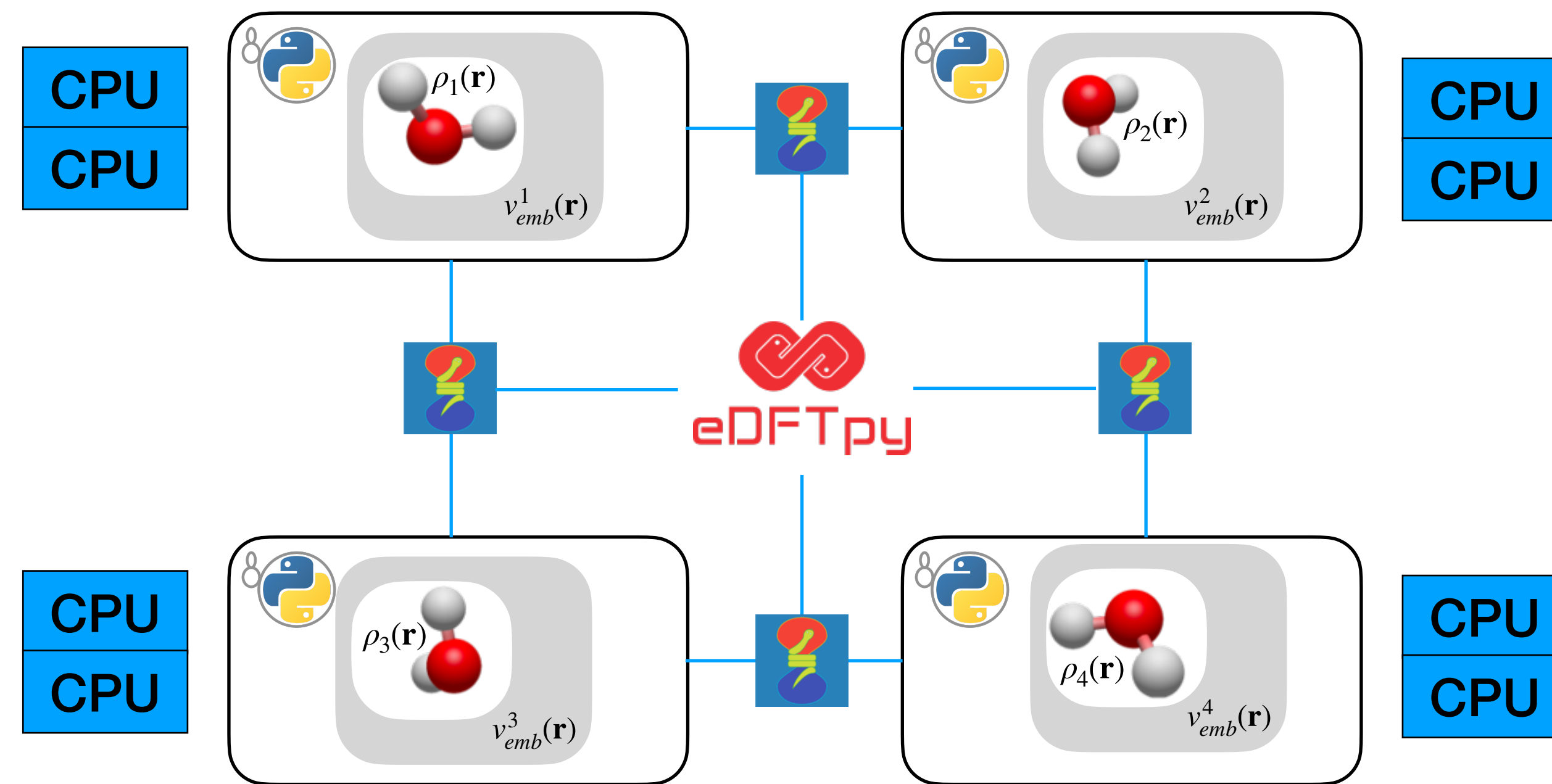
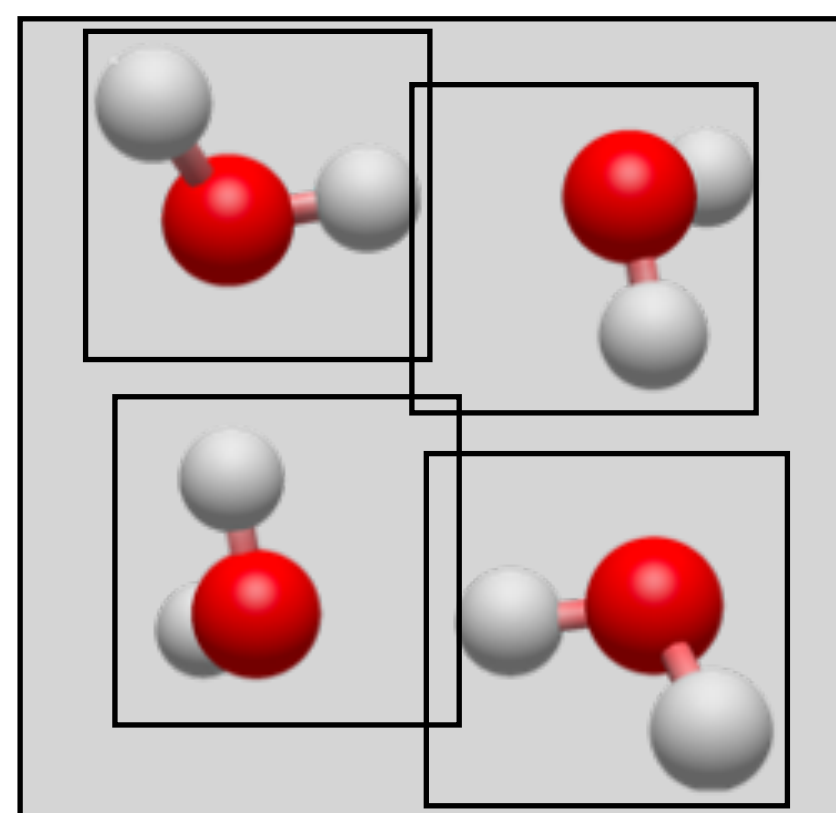
$$v_{xc}^{nad}[\{\rho_I\}](\mathbf{r})$$

eDFTpy: Divide and Conquer!



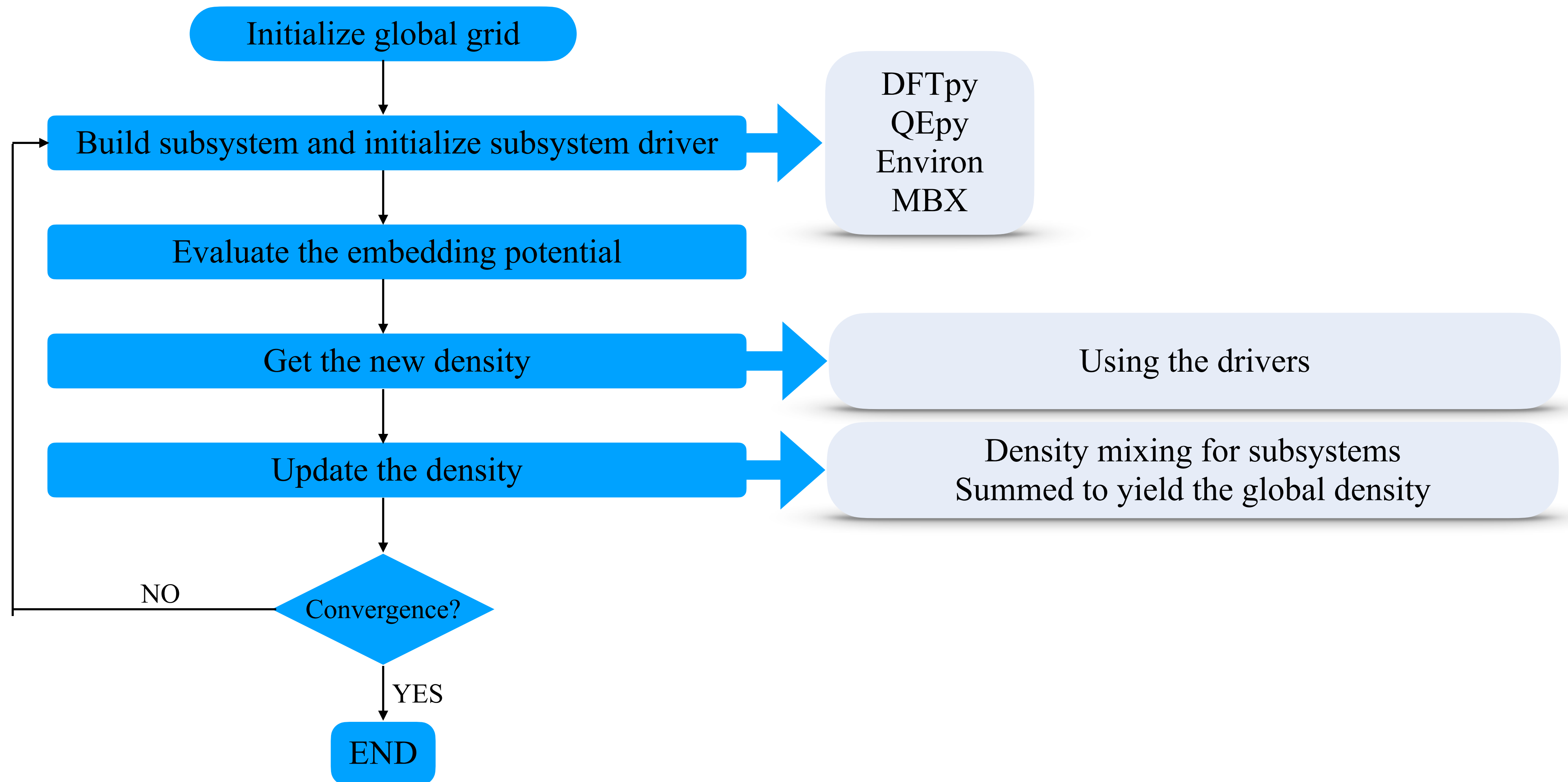
eDFTpy: Divide and Conquer!

- Subsystems are assigned their own set of CPUs and simulation cell
- Global potentials (such as v_H) need to be computed on the global grid

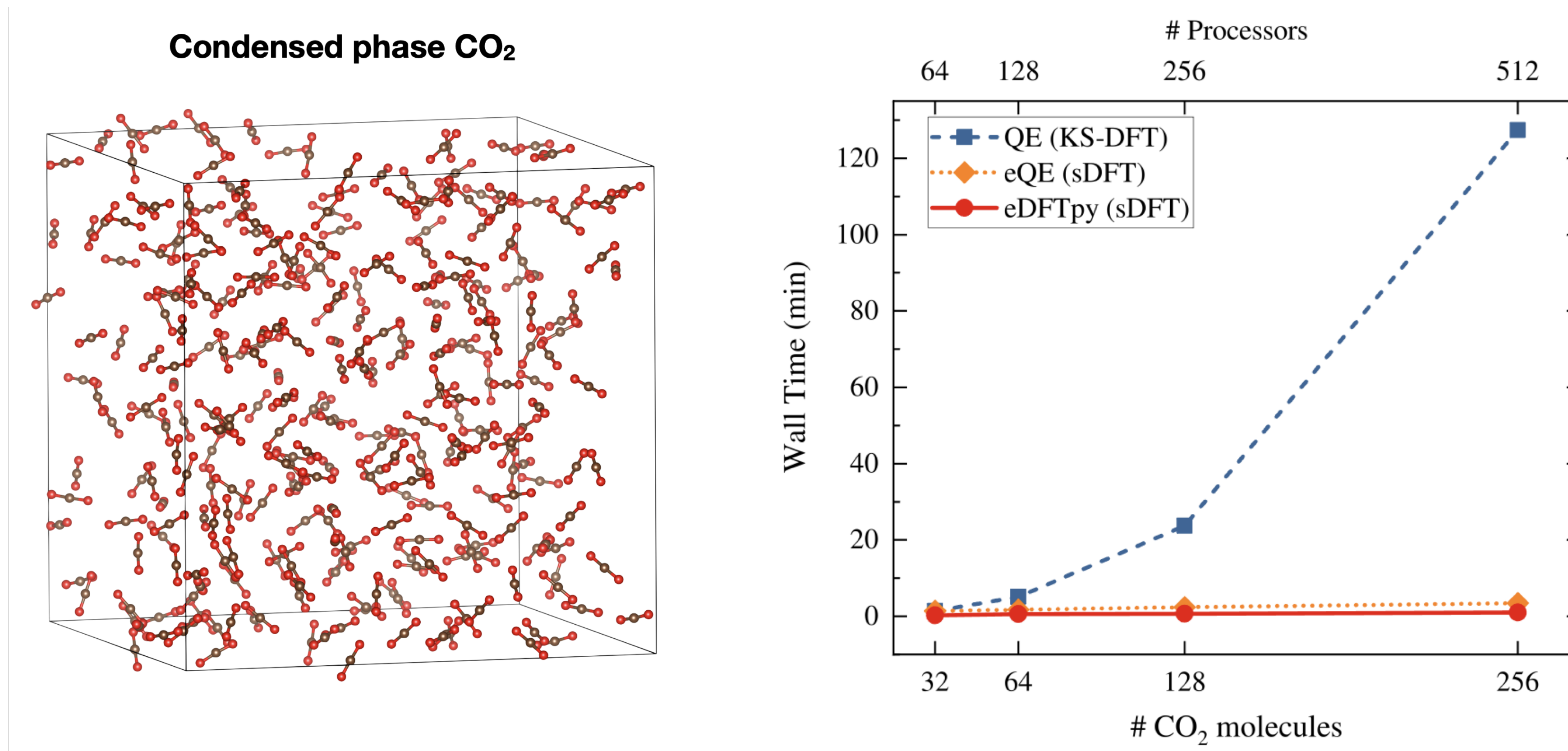


- Large simulation cell (physical)
- Subsystem cells (not physical)
- eDFTpy handles maps between them

eDFTpy: Divide and Conquer!

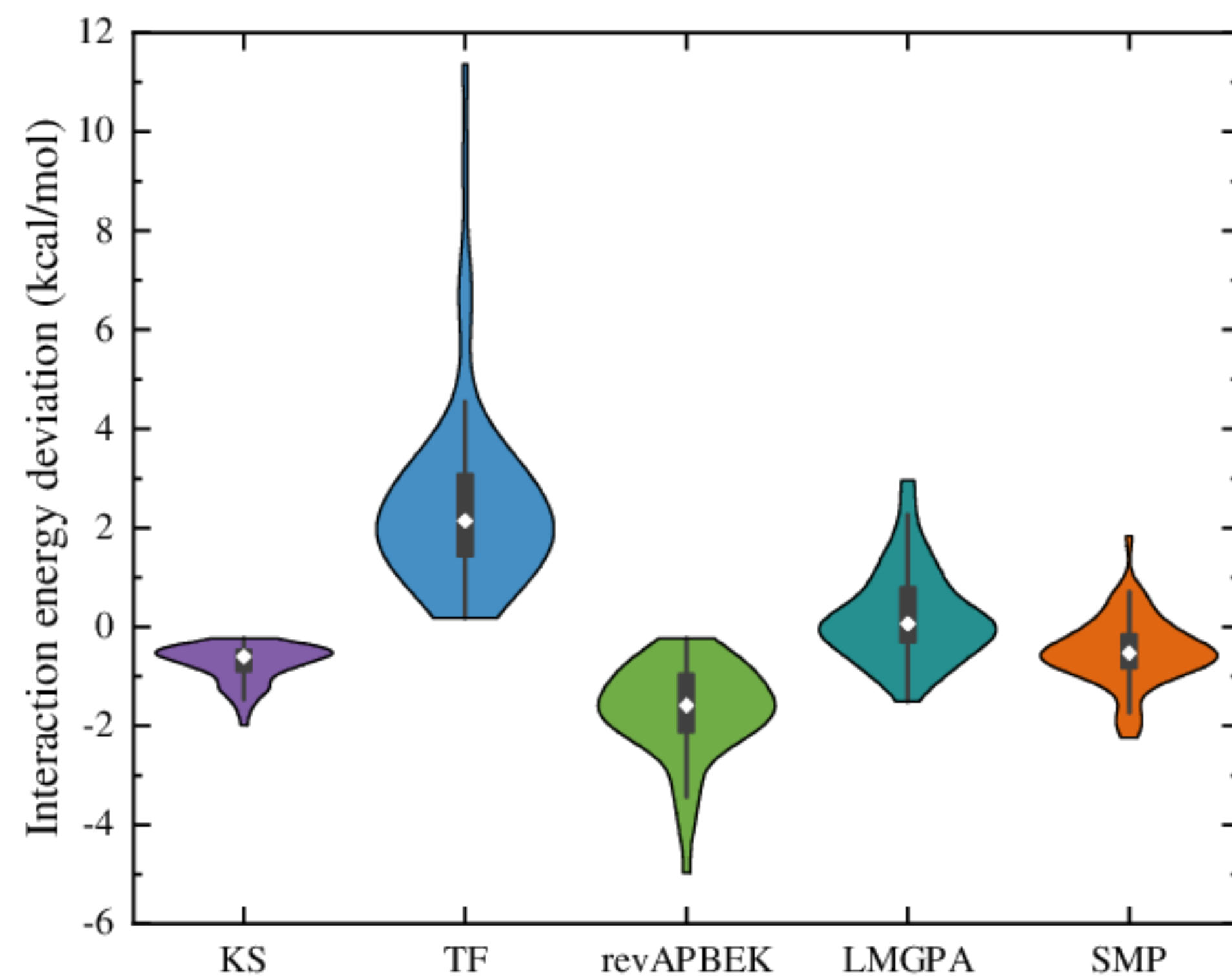


What can we do with sDFT?

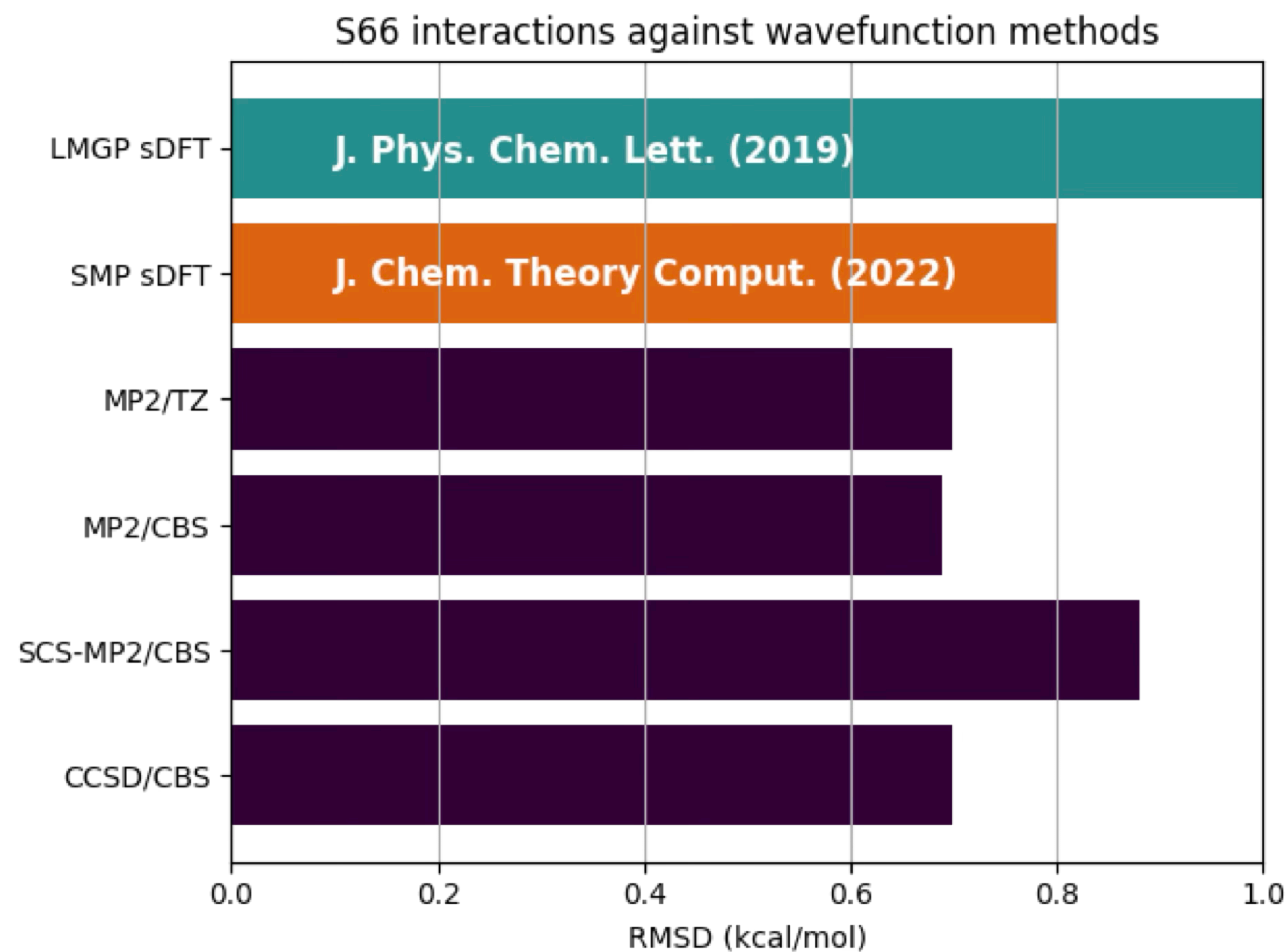


- Comp. Phys. Comm. **269**, 108122 (2021)
- Int. J. Quantum Chem. **117** e25401 (2017)
- J. Phys. Chem. Lett. **10** 7554 (2019)

How accurate is sDFT?



- Comp. Phys. Comm. **269**, 108122 (2021)
- Int. J. Quantum Chem. **117** e25401 (2017)
- J. Phys. Chem. Lett. **10** 7554 (2019)

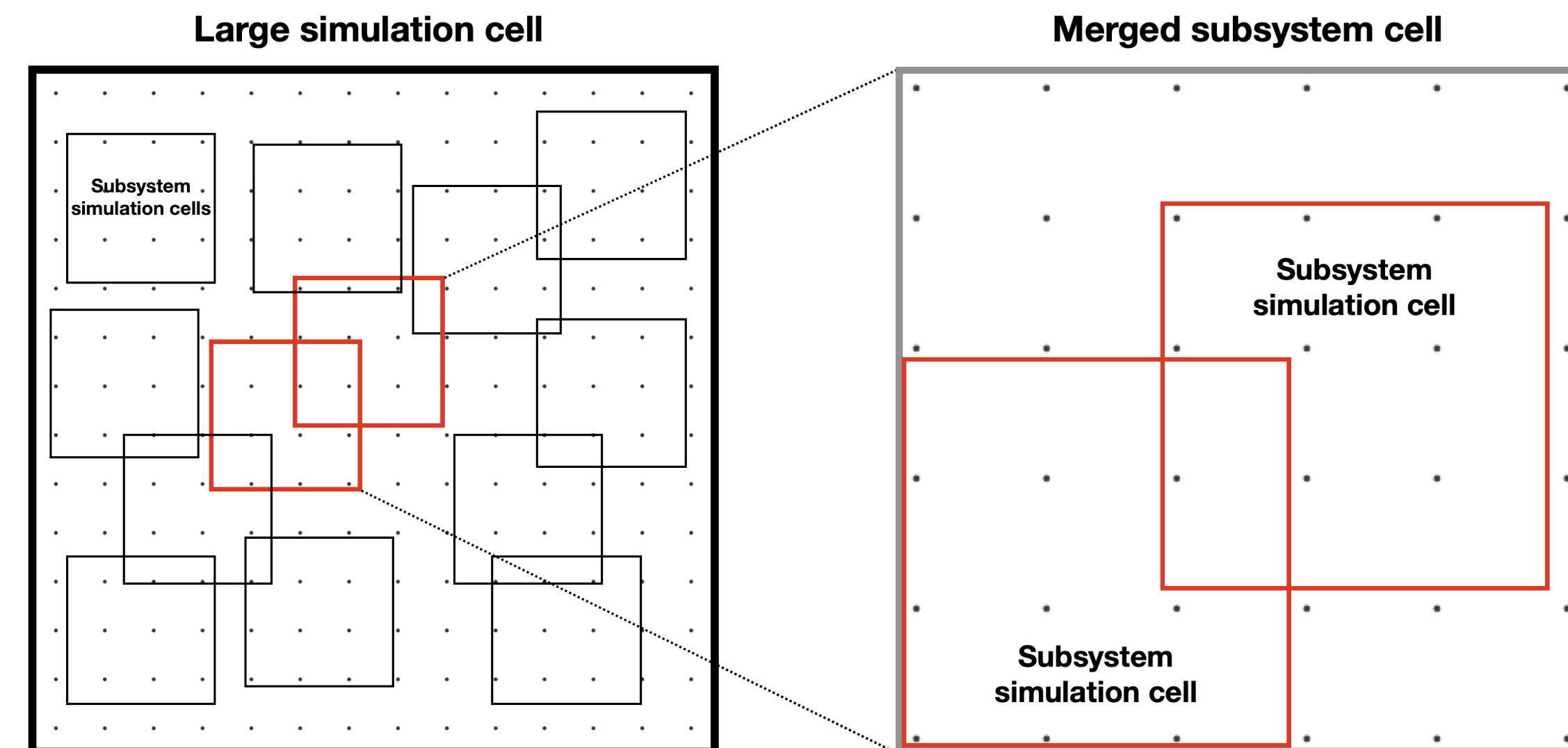
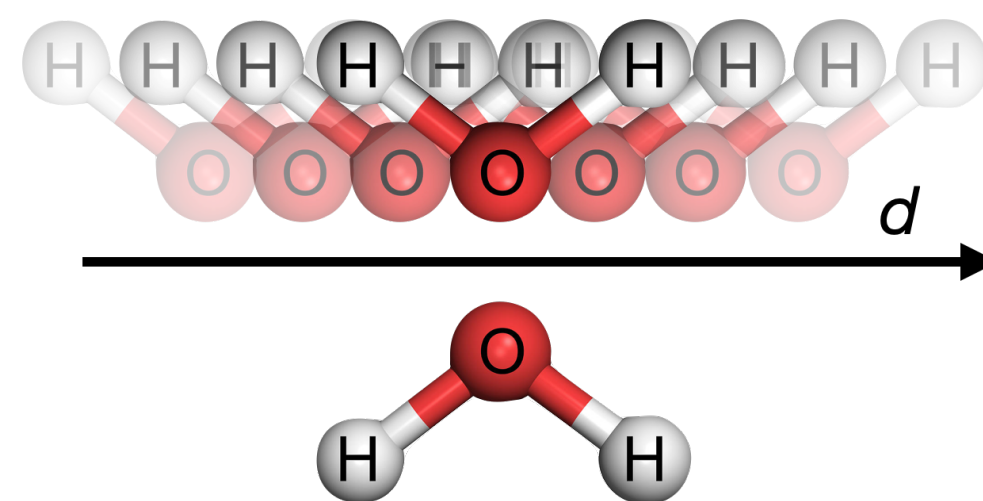
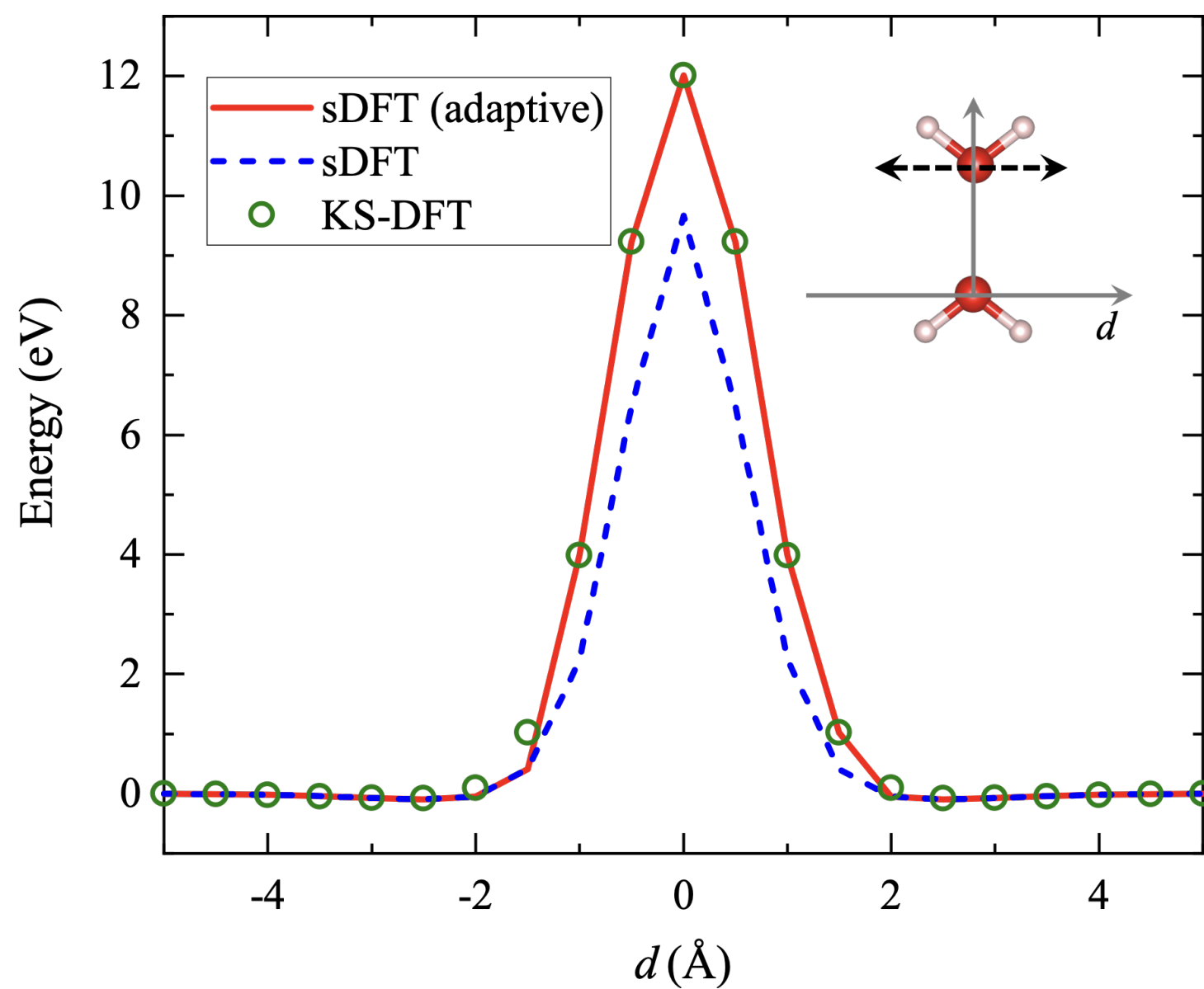


- J. Chem. Theory Comput. **7** 2427 (2011)

Even we can do it: Dynamically

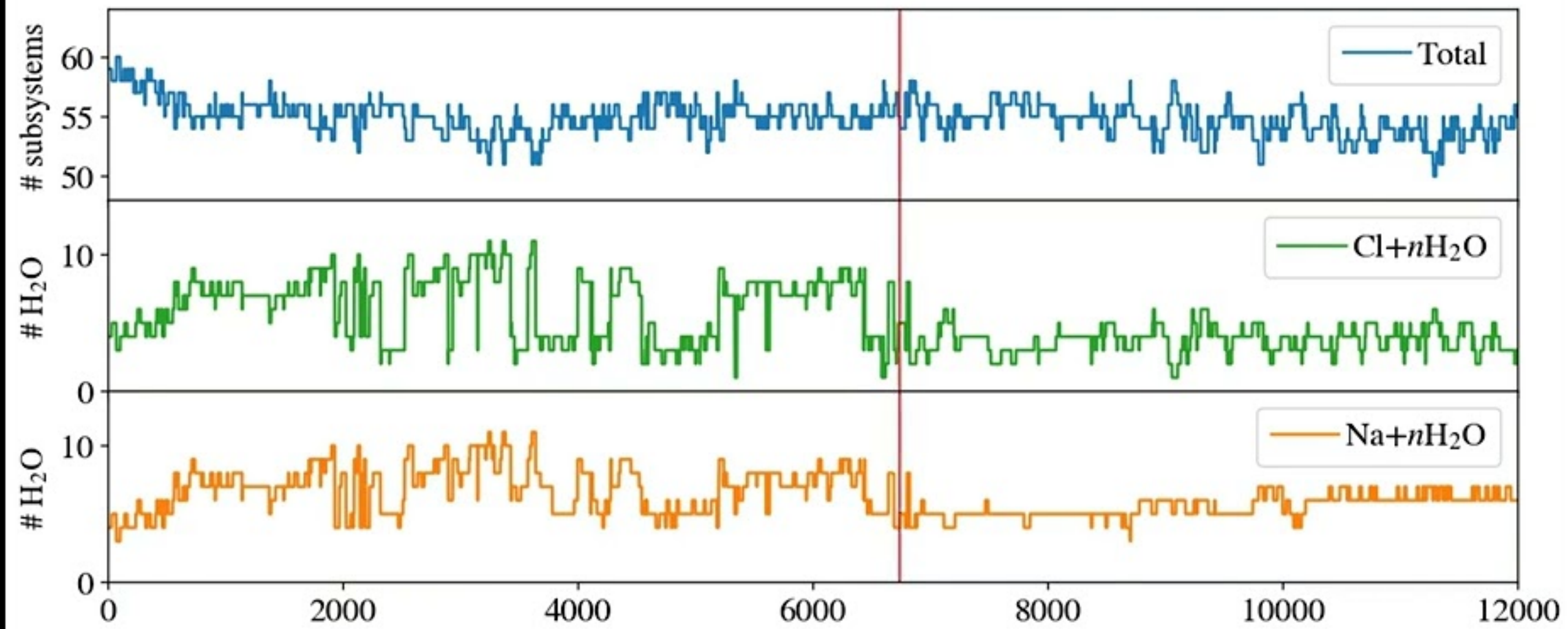
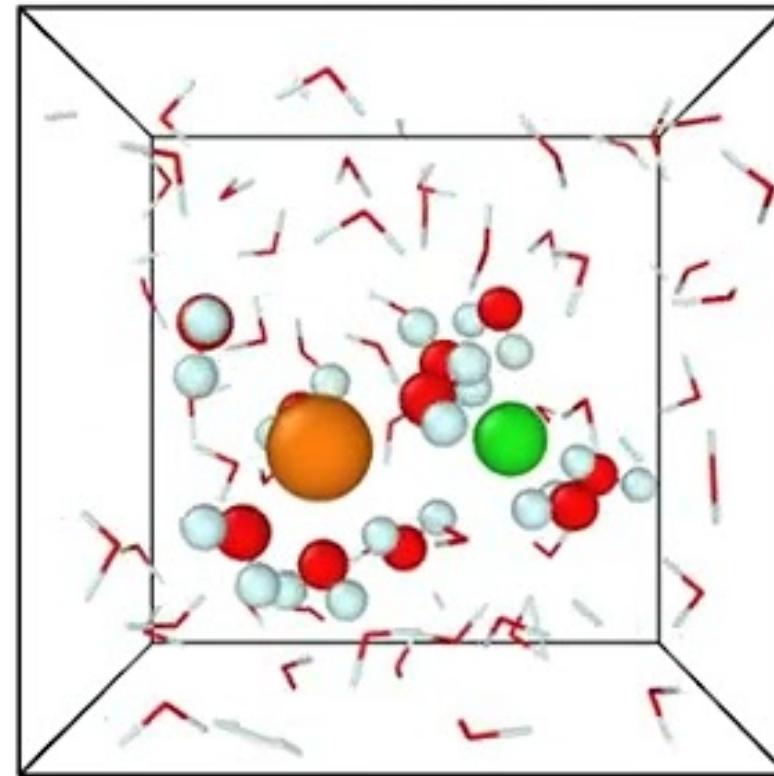
- The nonadditive kinetic energy ($T_s^{nadd}[\rho]$), is not accurate enough when...
 - ...there are covalent bonds between subsystems
 - ...subsystems are too close

Adaptive Subsystem Density Functional Theory



- Adaptive computational settings
- Adaptive MPI communicators
- Adaptive data allocation

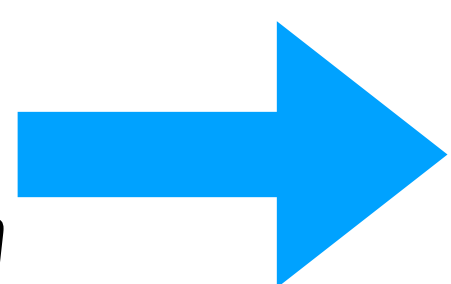
Even we can do it:
Dynamically



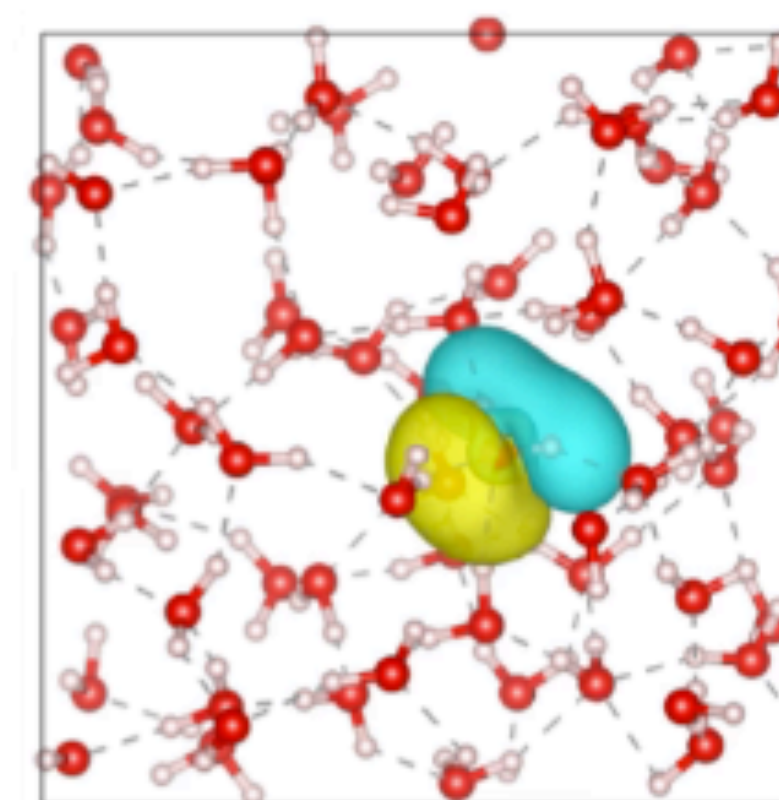
eDFTpy: Hands-on!

Polarization density (defined as the difference of the **embedded** and **isolated** molecular electron density) of an **embedded water molecule** in a 64 WATER system

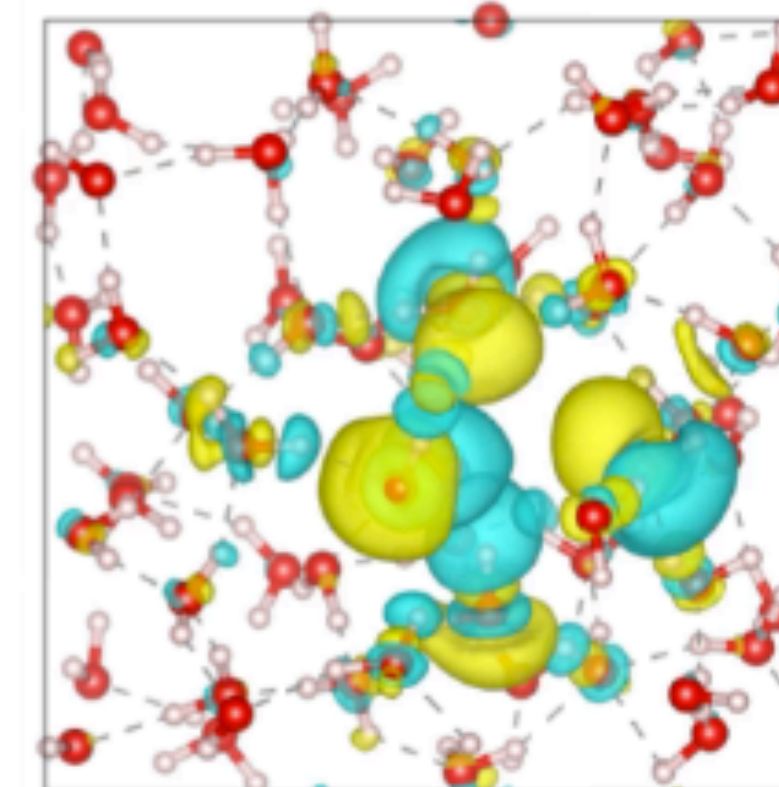
1. Run density Optimization
2. Get Polarization Densities!



Run the Tutorial



$$\rho_A^{pol} = \rho_A^{emb} - \rho_A^{iso}$$



$$\rho_B^{pol} = \rho_B^{emb} - \rho_B^{iso}$$

eDFTpy: Hands-on!

Open JN

1) Open the interactive expanse

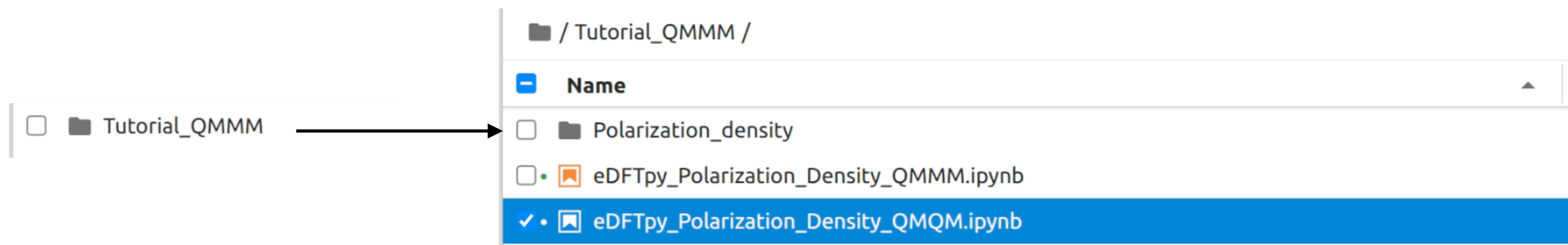
2) pwd to check you're in your working directory

```
/expanse/lustre/projects/csd973/your user name here
```

3) Copy the work directory:

```
cp -r /expanse/lustre/projects/csd973/jmartinez2/Tutorial_QMMM/ .
```

4) Check the JN inside the folder:

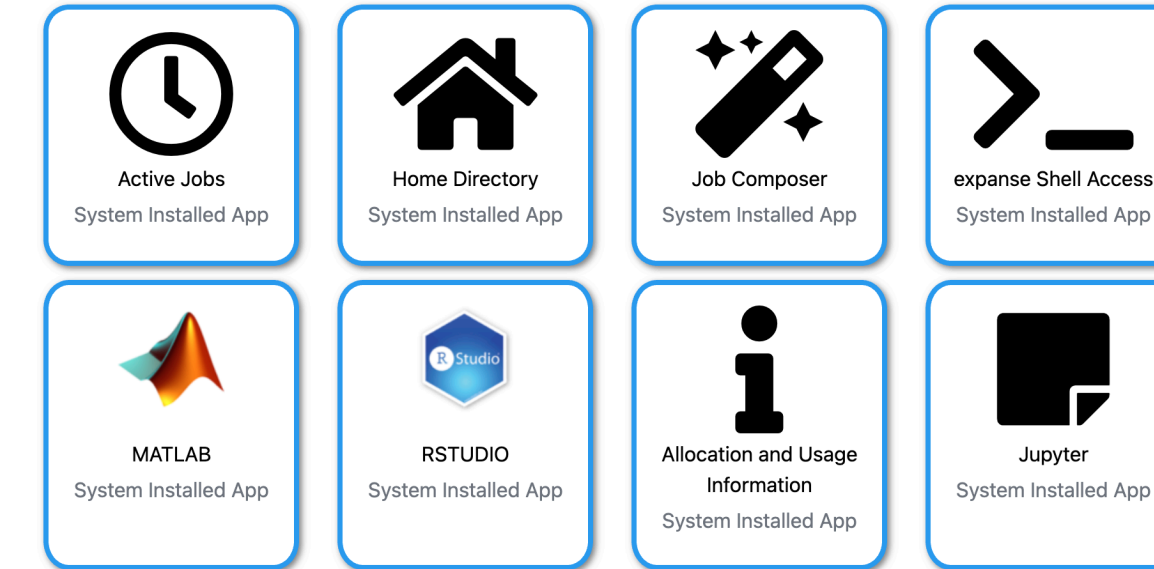


SDSC

The Expanse portal provides an integrated, and easy to use interface to access Expanse HPC resource.

With the portal, researchers can manage files (create, edit, move, upload, and download), view job templates for various applications, submit and monitor jobs, run interactive applications, and connect via SSH. The portal has no end-user installation requirements other than access to a modern up-to-date web browser

Pinned Apps A featured subset of [all available apps](#)



eDFTpy: Hands-on!

Run the Tutorial

jupyter eDFTpy_Polarization_Density_QMQM Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help

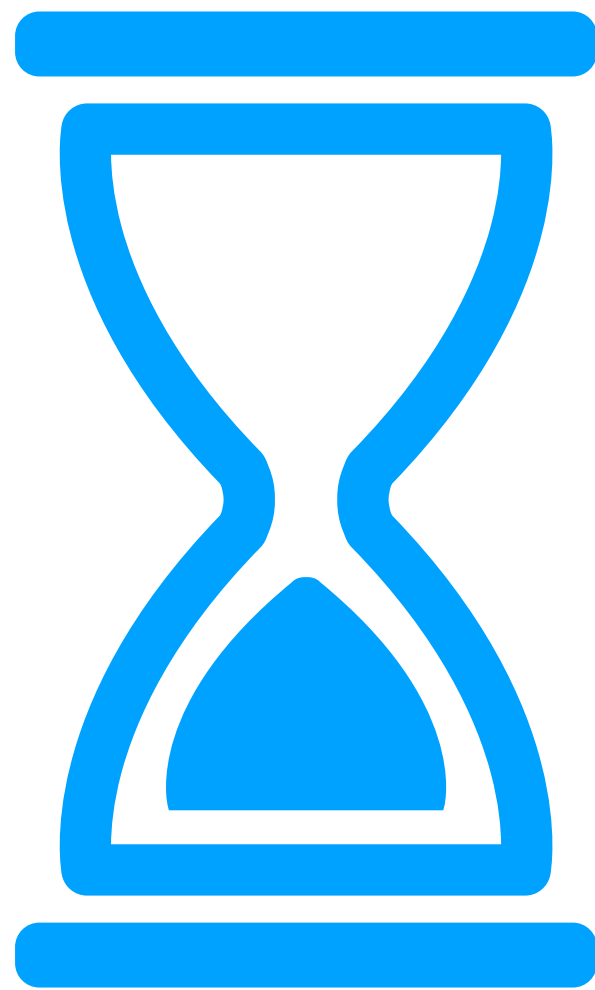
Truste

JupyterLab edftpy_qmmm

Polarization Density: Water embedded in Water using sDFT

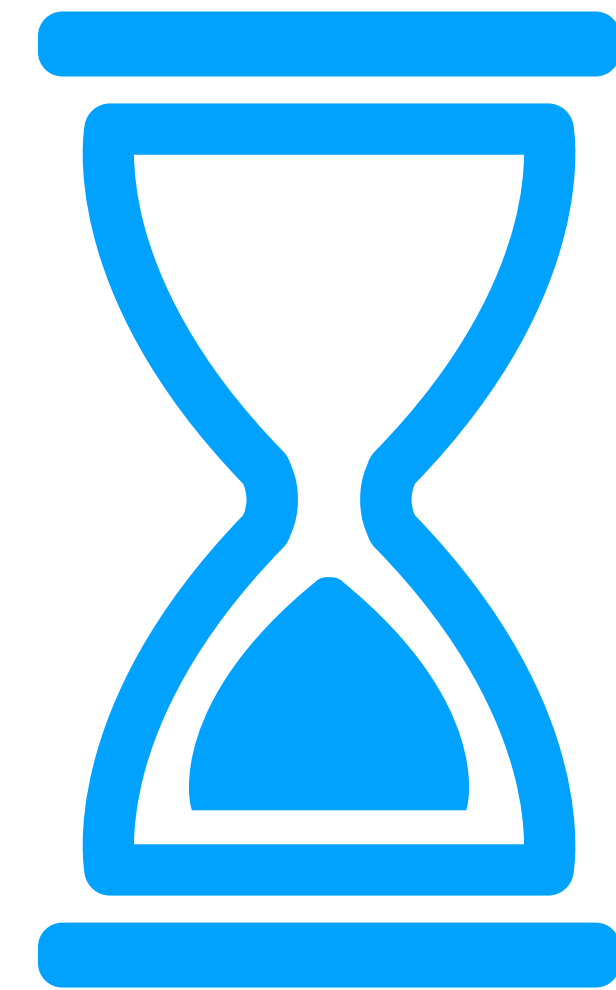
```
[1]: import os
import numpy as np
import glob
from ase.io import read, write
from ase.io.xsf import read_xsf, write_xsf
import matplotlib.pyplot as plt
from ipywidgets import interact, IntSlider
```

Think about some interesting systems that can be modeled with sDFT!



- Molecular condensed phases !
- Anything ASE can do sDFT can do!

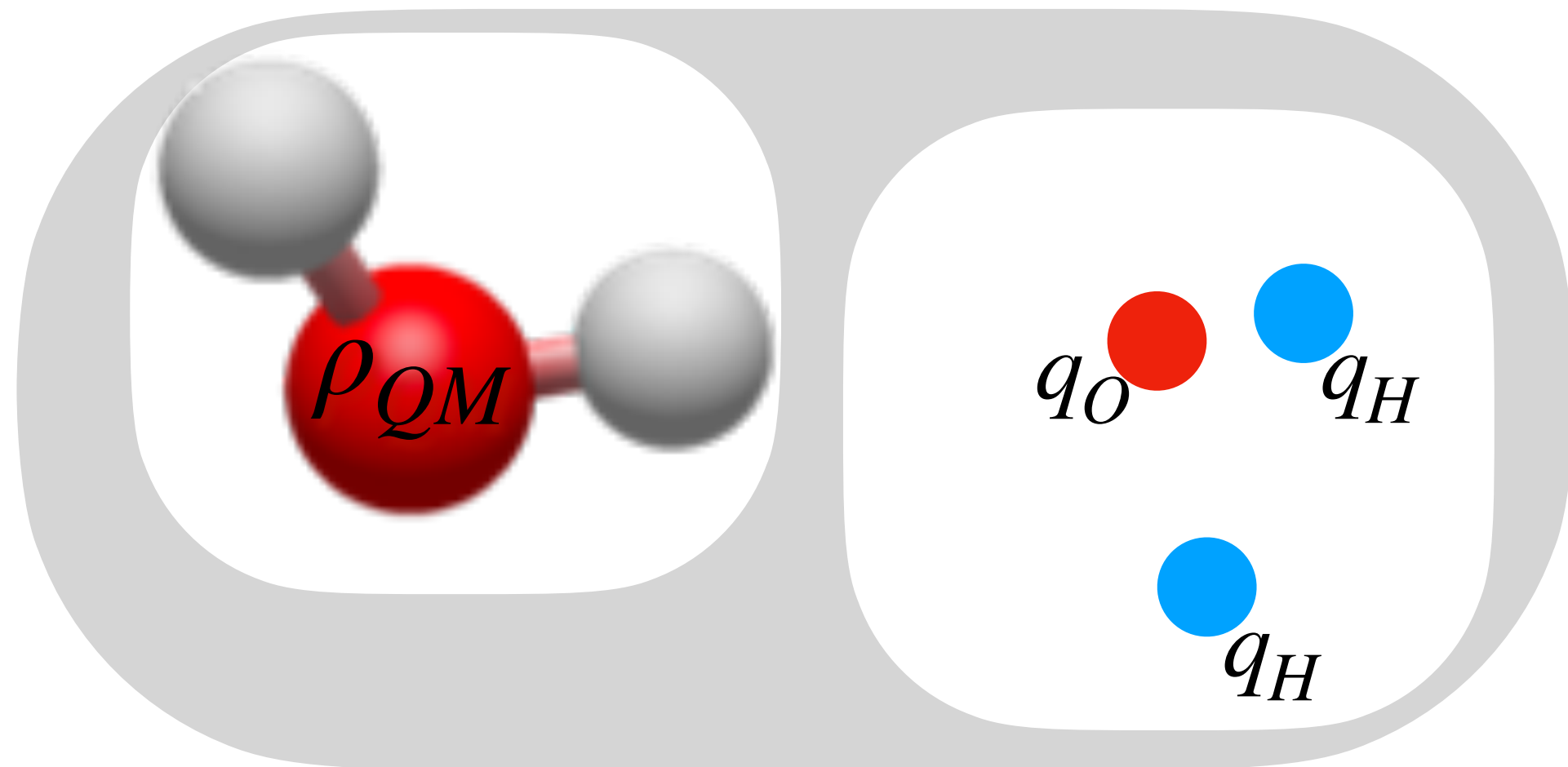
Let's discuss after the break!



We can also use a QM/MM approach!

Mixing eDFTpy with MBX

- Split system into (two) subsystems:

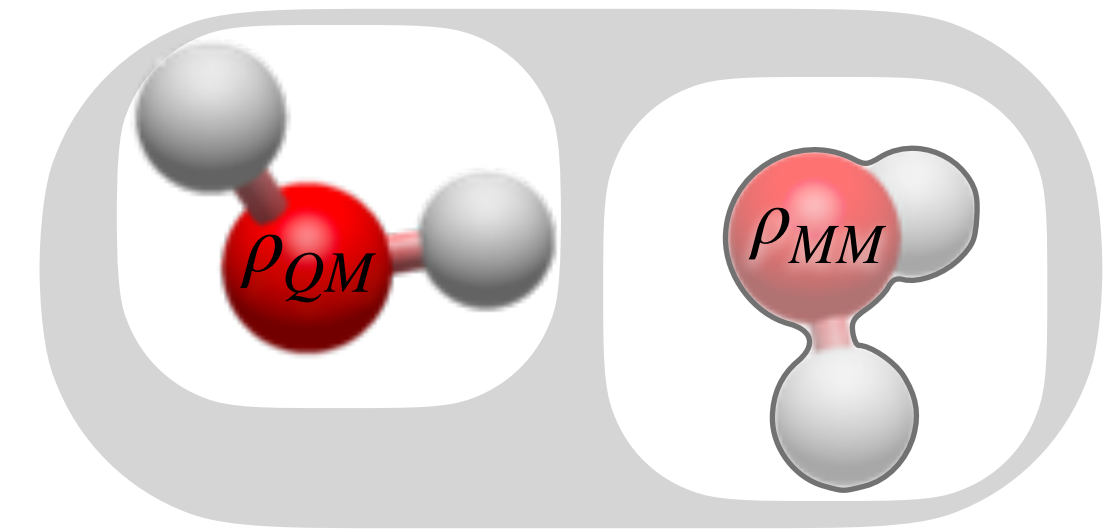


Not accurate enough!

do not faithfully represent any electronic structure!

We can also use a QM/MM approach!

- Split system into (two) subsystems: $\rho(\mathbf{r}) = \rho_{QM}(\mathbf{r}) + \rho_{MM}(\mathbf{r})$



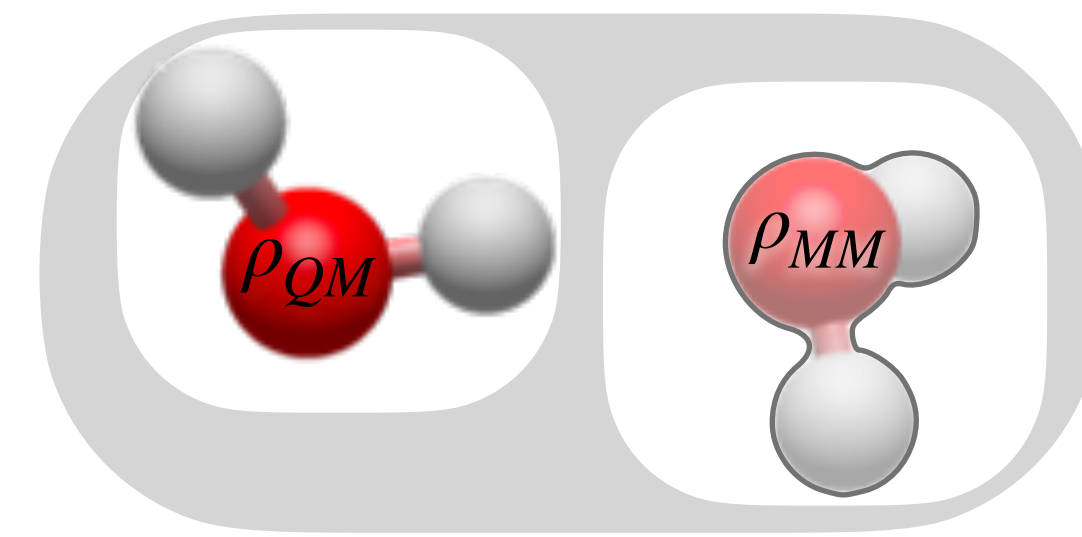
- Total Energy: $E[\rho_{QM}, \rho_{MM}] = E[\rho_{QM}] + E[\rho_{MM}] + E^{nad}[\rho_{QM}, \rho_{MM}]$

$$E[\rho_{QM}] = T_s[\rho_{QM}] + E_H[\rho_{QM}] + E_{xc}[\rho_{QM}] + \int v_{QM}(\mathbf{r})\rho_{QM}(\mathbf{r})d\mathbf{r}$$

$$E[\rho_{MM}] = T_s[\rho_{MM}] + E_H[\rho_{MM}] + E_{xc}[\rho_{MM}] + \int v_{MM}(\mathbf{r})\rho_{MM}(\mathbf{r})d\mathbf{r}$$

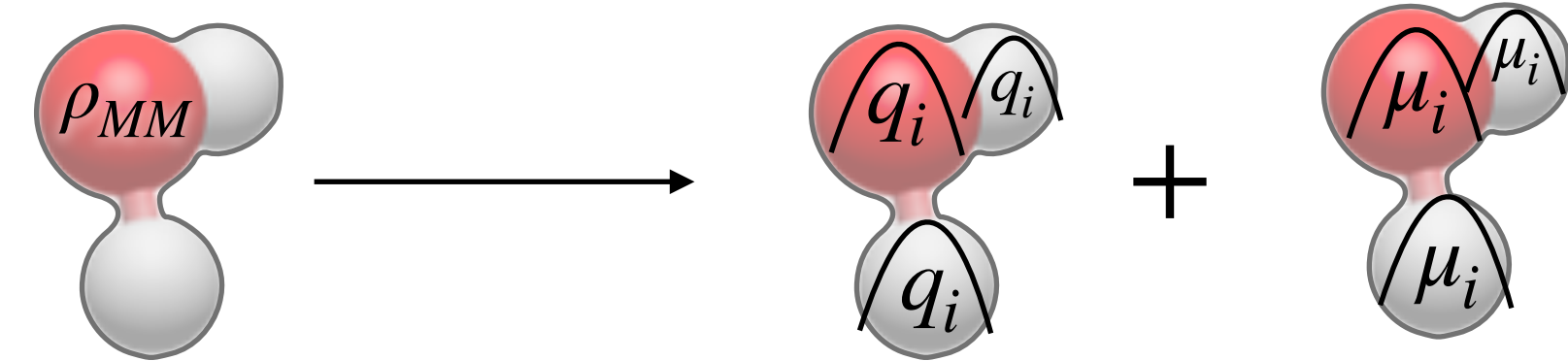
$$E^{nad}[\rho_{QM}, \rho_{MM}] = T_s^{nad}[\rho_{QM}, \rho_{MM}] + E_{xc}^{nad}[\rho_{QM}, \rho_{MM}] + \int \frac{\rho_{QM}(\mathbf{r})\rho_{MM}(\mathbf{r})}{|\mathbf{r} - \mathbf{r}'|}d\mathbf{r}d\mathbf{r}' + \int \rho_{MM}(\mathbf{r})v_{QM}(\mathbf{r})d\mathbf{r} + \int \rho_{QM}(\mathbf{r})v_{MM}(\mathbf{r})d\mathbf{r}$$

QM/MM



- Key: Density MM

$$\rho_{MM}(\mathbf{r}) = \sum_{i \in MMcharges} \rho_{q_i}(\mathbf{r}) + \sum_{j \in MMdipoles} \rho_{\mu_j}(\mathbf{r})$$



- Now we have two Embedding Potentials!

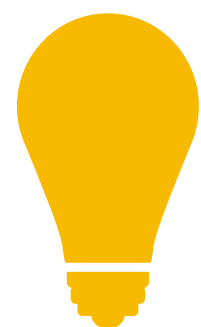
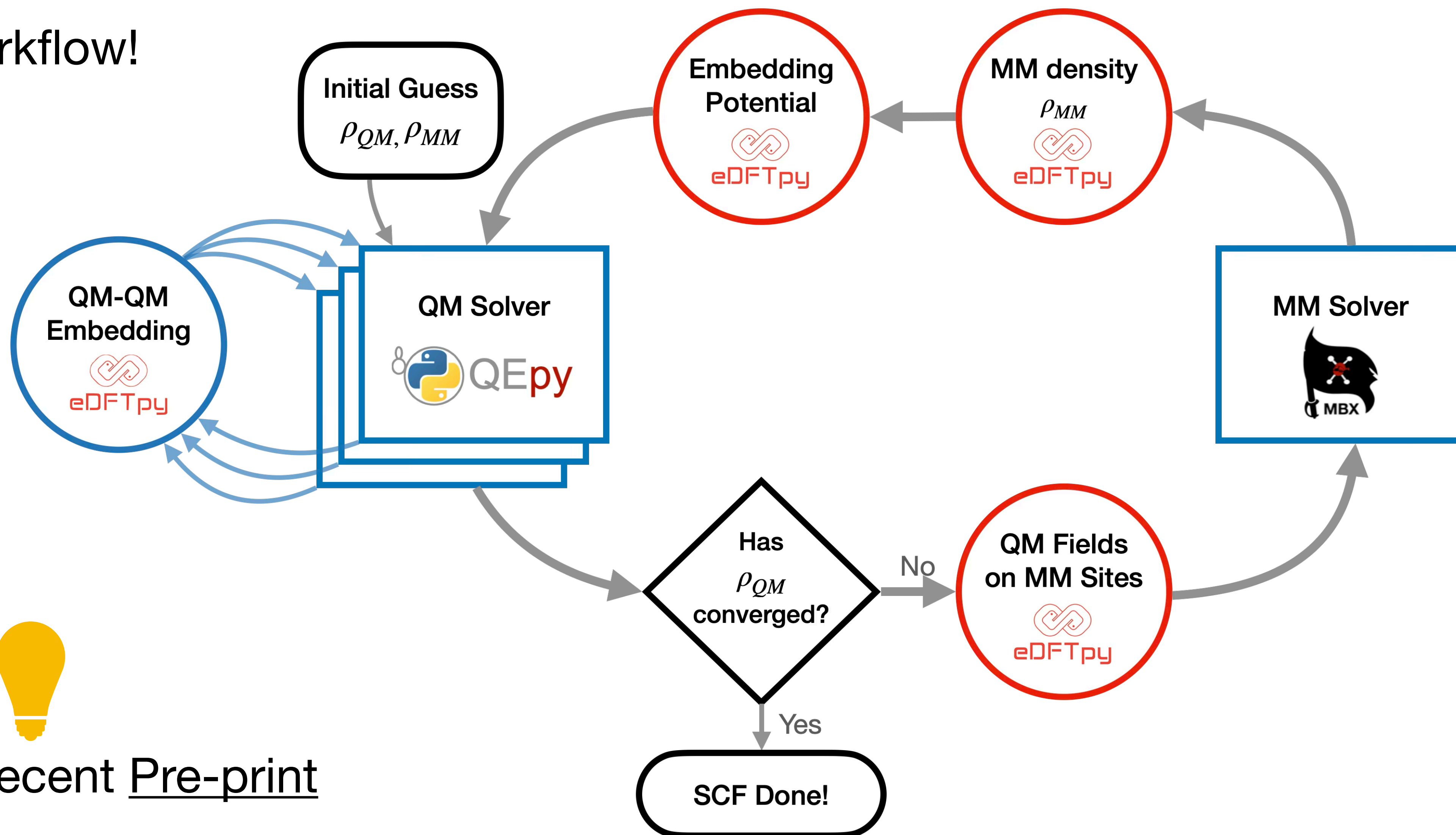
$$v_{emb}^{QM}(\mathbf{r}) = \frac{\delta E^{nad}[\rho_{QM}, \rho_{MM}]}{\delta \rho_{QM}(\mathbf{r})} = v_{MM}(\mathbf{r}) + \int d\mathbf{r}' \frac{\rho_{MM}(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} + \frac{\delta T_s^{nad}}{\delta \rho_{QM}(\mathbf{r})} + \frac{\delta E_{xc}^{nad}}{\delta \rho_{QM}(\mathbf{r})}$$

$$v_{emb}^{MM}(\mathbf{r}) = \frac{\delta E^{nad}[\rho_{QM}, \rho_{MM}]}{\delta \rho_{MM}(\mathbf{r})} = v_{QM}(\mathbf{r}) + \int d\mathbf{r}' \frac{\rho_{QM}(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} + \frac{\delta T_s^{nad}}{\delta \rho_{MM}(\mathbf{r})} + \frac{\delta E_{xc}^{nad}}{\delta \rho_{MM}(\mathbf{r})}$$

QM/MM

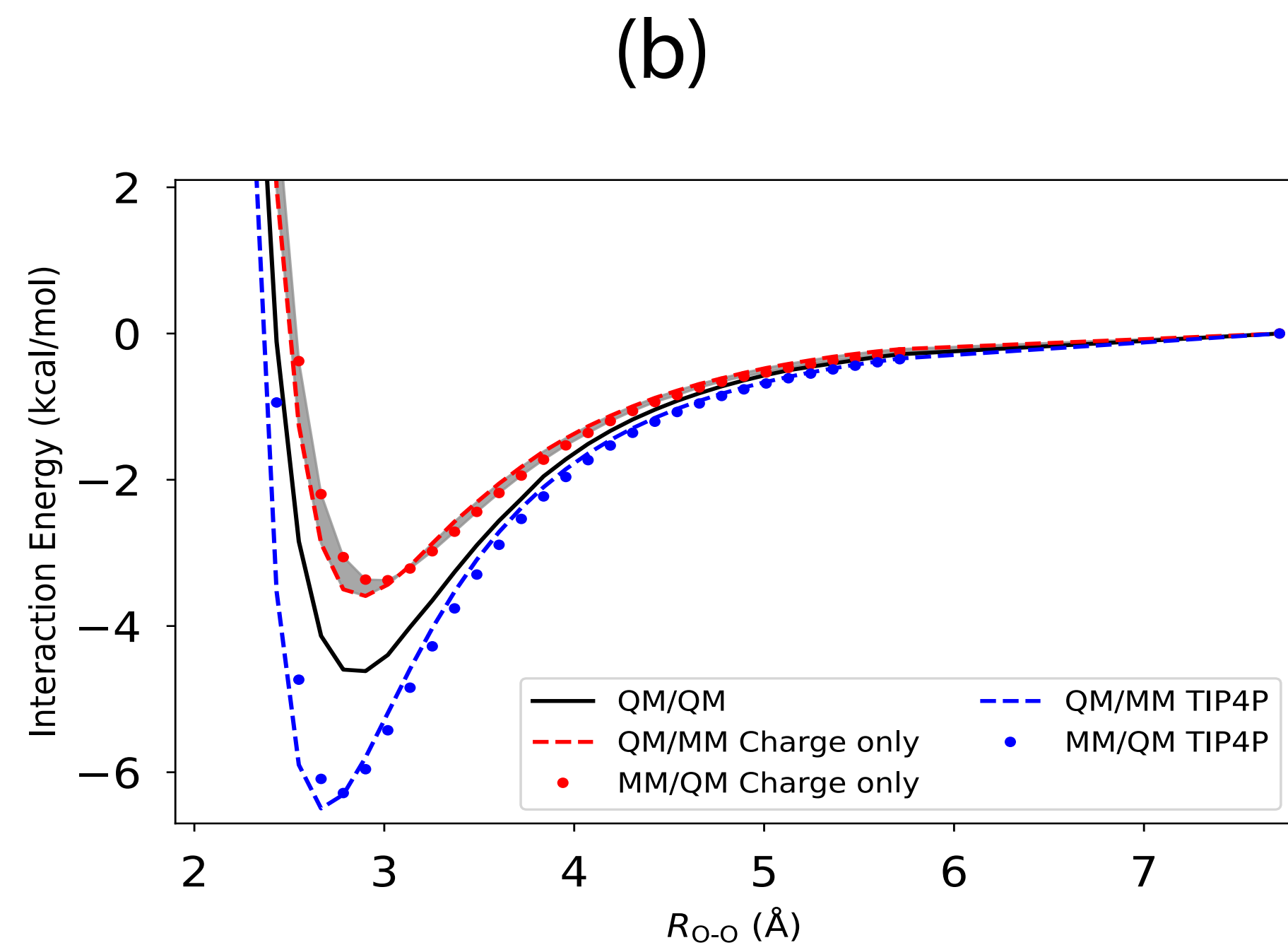
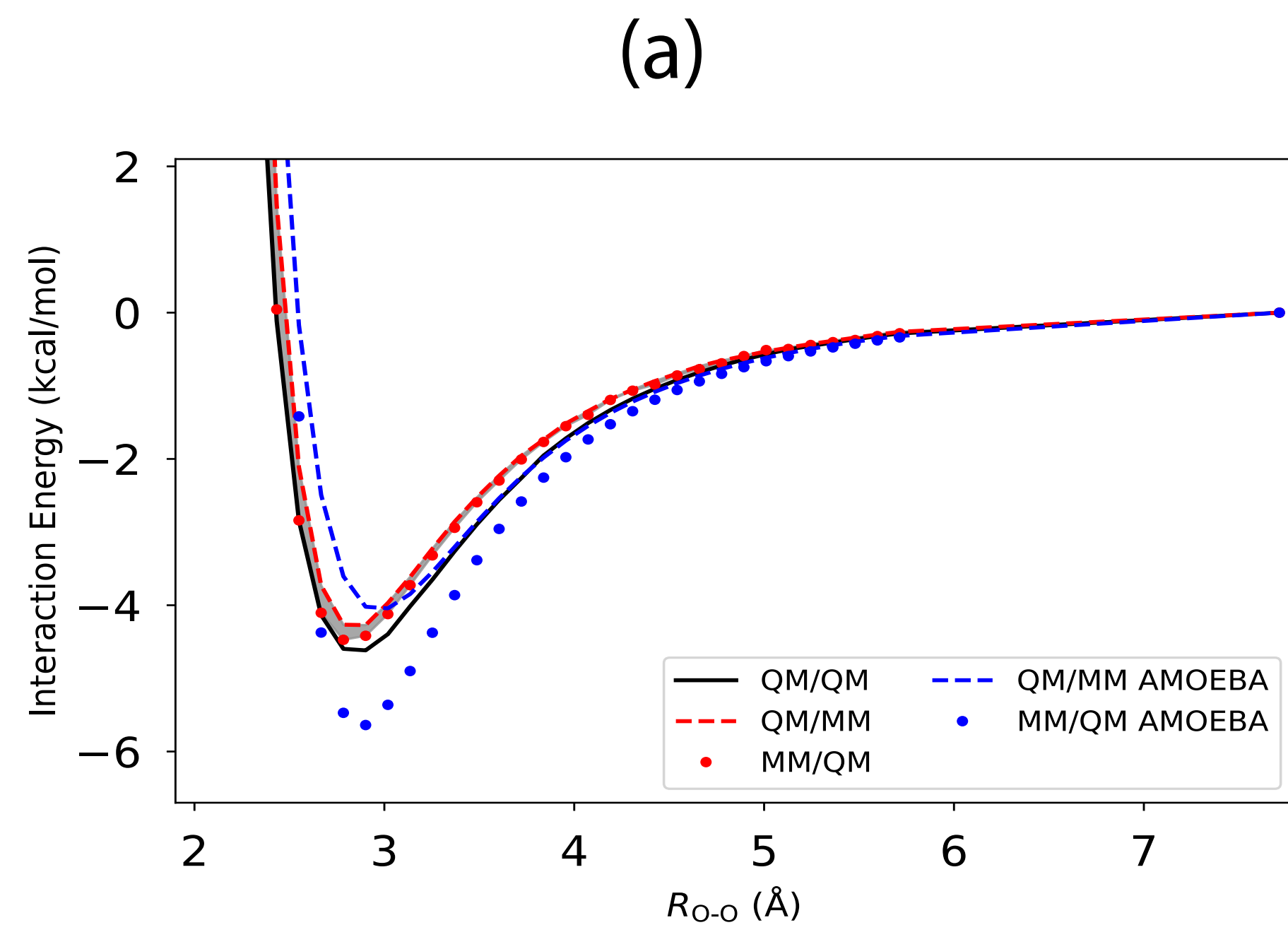
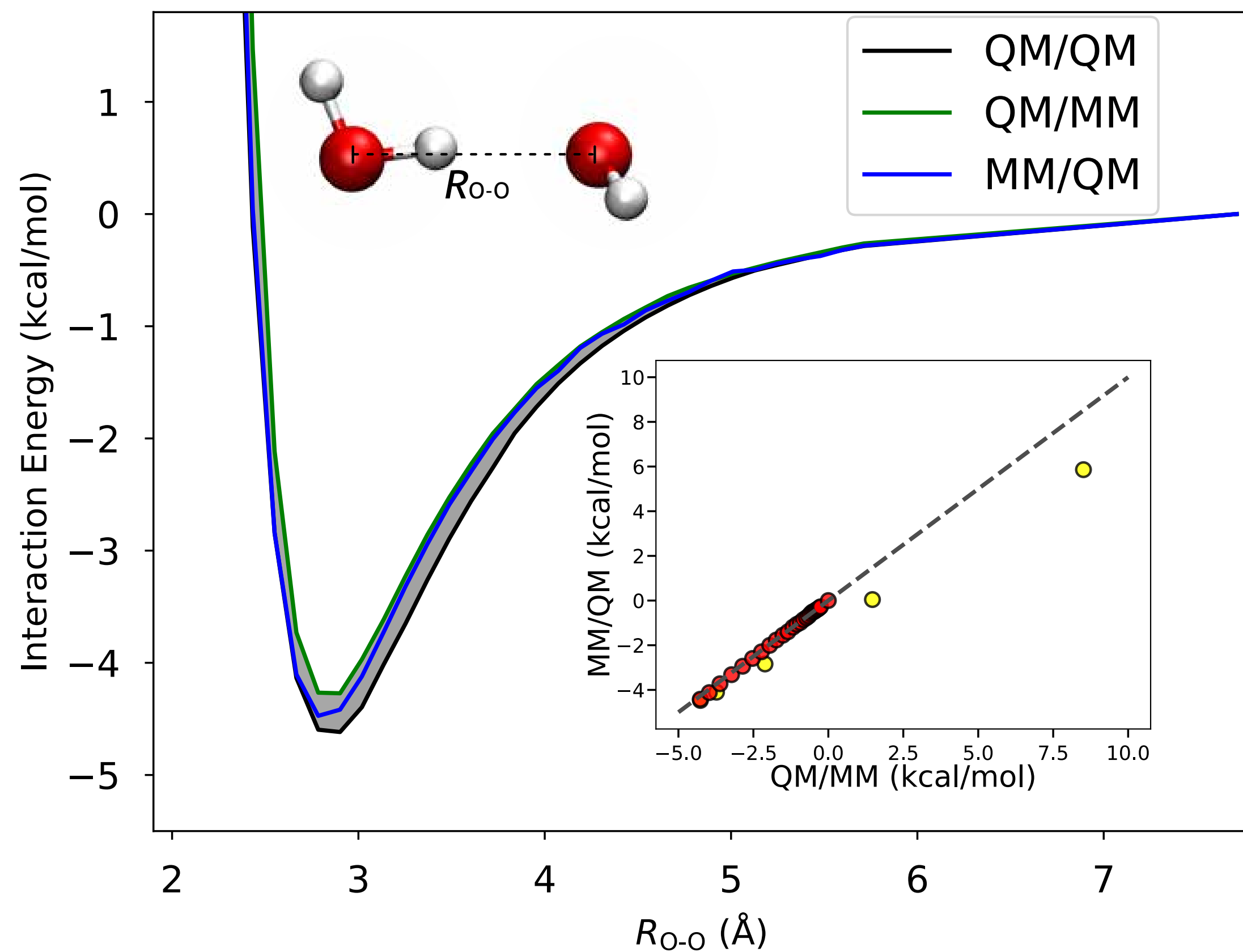
$$\rho_{q_i}(\mathbf{r}) = (N_i - q_i)g_{\sigma_i}(\mathbf{r} - \mathbf{R}_i)$$
$$\rho_{\mu_j}(\mathbf{r}) = -\vec{\mu}_j \cdot \vec{\nabla} g_{\sigma_j}(\mathbf{r} - \mathbf{R}_j)$$

- General workflow!



Check our recent Pre-print

QM/MM: How good is our QM/MM implementation?



eDFTpy: Hands-on!

Open JN

1) Open the interactive expanse

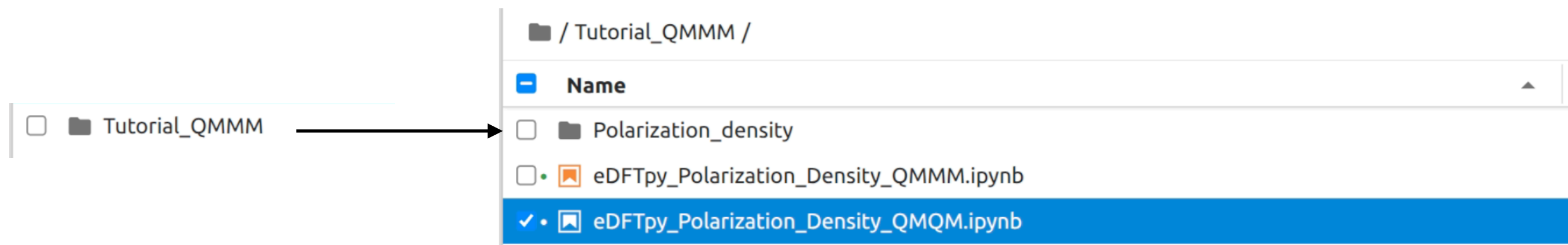
2) pwd to check you're in your working directory

```
/expanse/lustre/projects/csd973/your user name here
```

3) Copy the work directory:

```
cp -r /expanse/lustre/projects/csd973/jmartinez2/Tutorial_QMMM/ .
```

4) Check the JN inside the folder:

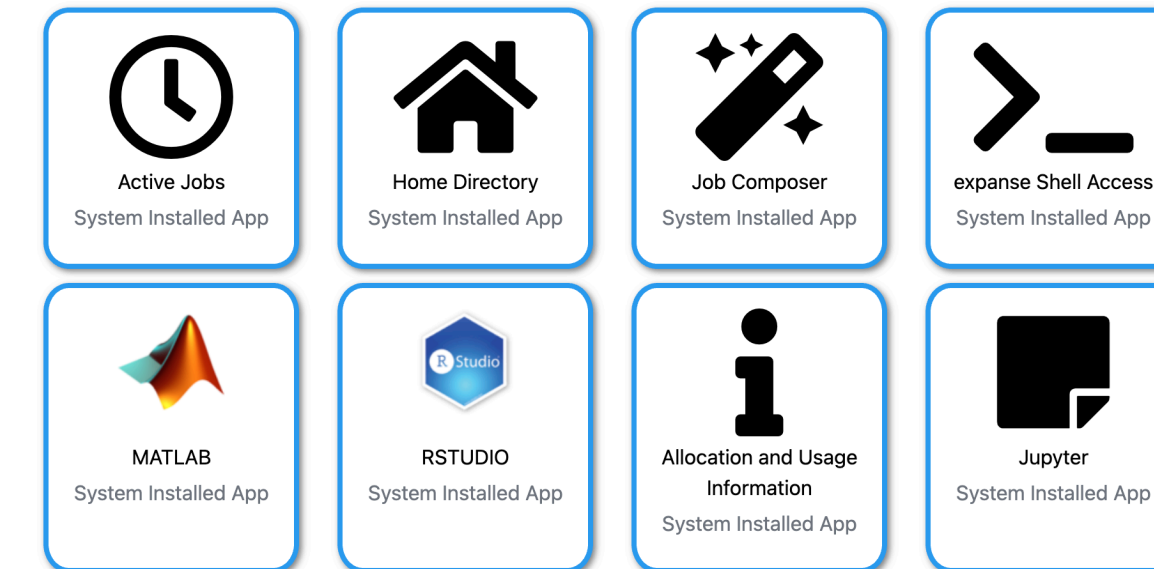


SDSC

The Expanse portal provides an integrated, and easy to use interface to access Expanse HPC resource.

With the portal, researchers can manage files (create, edit, move, upload, and download), view job templates for various applications, submit and monitor jobs, run interactive applications, and connect via SSH. The portal has no end-user installation requirements other than access to a modern up-to-date web browser

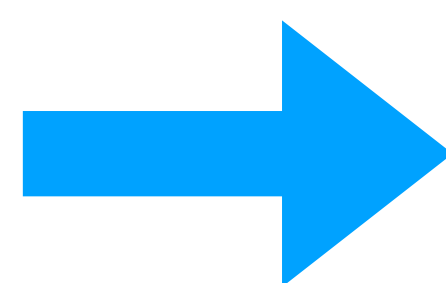
Pinned Apps A featured subset of [all available apps](#)



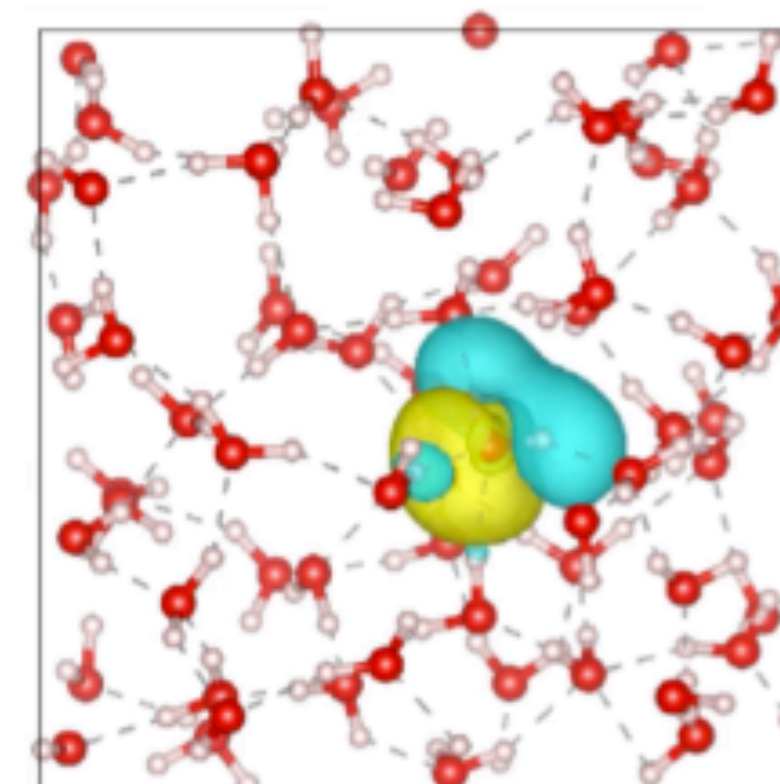
QM/MM

Polarization density (defined as the difference of the embedded and isolated molecular electron density) of an embedded water molecule in a 64 WATER system

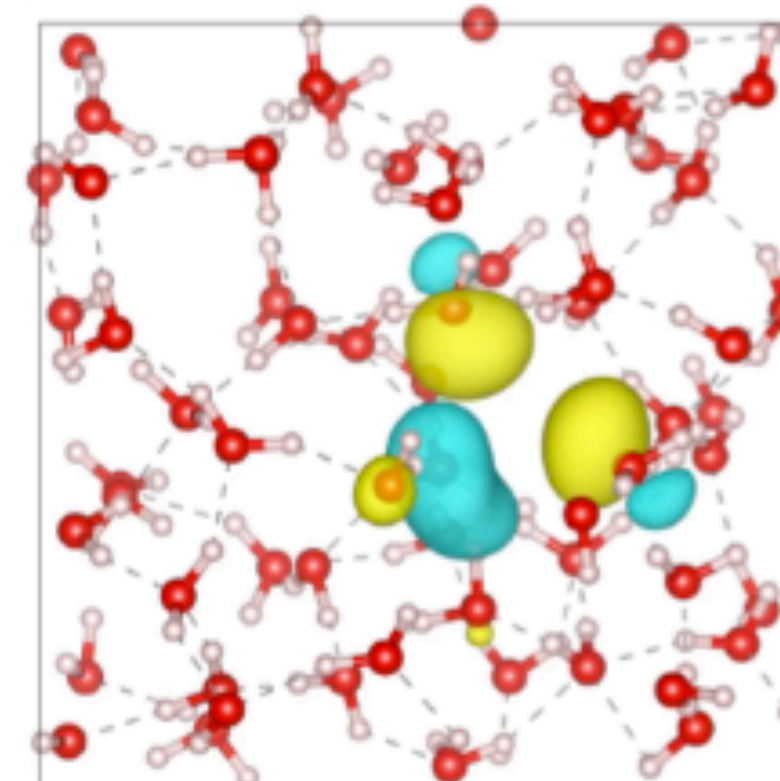
1. Run density Optimization
2. Get Polarization Densities!



Run the Tutorial



$$\rho_A^{pol} = \rho_A^{emb} - \rho_A^{iso}$$

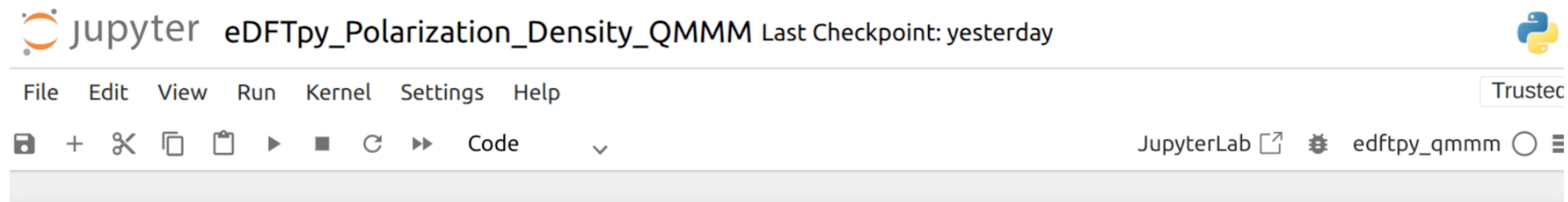
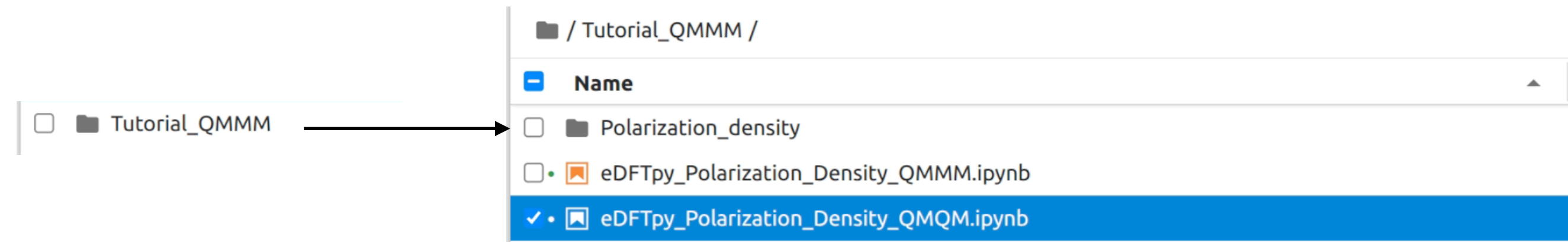


$$\rho_B^{pol} = \rho_B^{emb} - \rho_B^{iso}$$

eDFTpy: Hands-on!

Run the Tutorial

1) F5 or reload your JN in the web browser:



Polarization Density: Water embedded in Water using sDFT/MBX

```
[25]: import os
import numpy as np
import glob
from ase.io import read, write
from ase.io.xsf import read_xsf, write_xsf
import matplotlib.pyplot as plt
from ipywidgets import interact, IntSlider
```


sDFT vs QMMM challenge

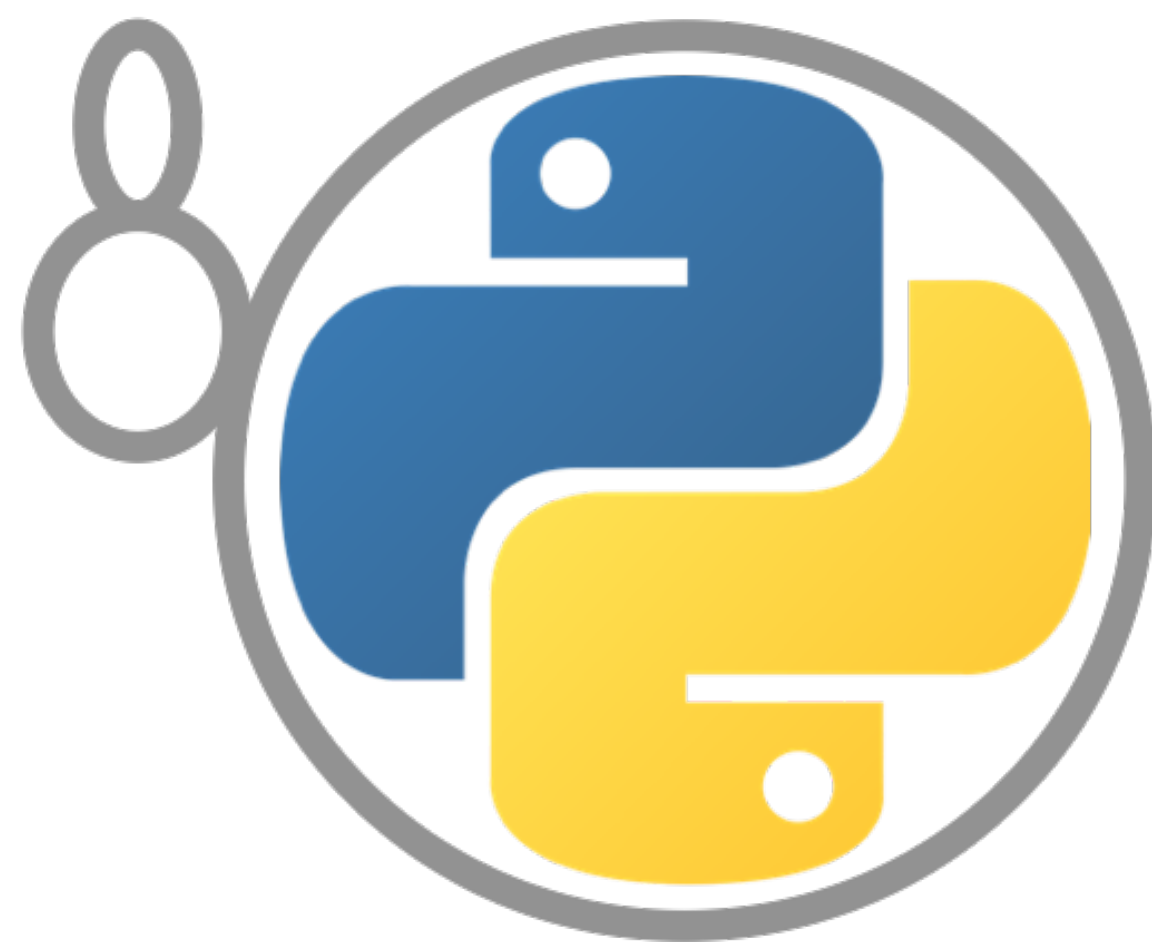
Run relaxation of NaCl in water using adaptive sDFT Tutorial

```
/exppanse/lustre/projects/csd973/your_user_name_here/Tutorial_QMMM/nac1_water
```

Run relaxation of Glucose using sDFT and QM/MM checking that the Energy and Forces converged.

```
/exppanse/lustre/projects/csd973/your_user_name_here/Tutorial_QMMM/glucose
```

More about PRG



QEPy



DFTpy

